

Complete CSS Master Guide

Comprehensive beginner-to-intermediate CSS reference with concepts and examples.

1. Introduction to CSS

Concept: CSS (Cascading Style Sheets) is used to style HTML elements and control layout, colors, spacing, and responsiveness.

Example:

```
h1 { color: blue; font-size: 32px; }
```

2. CSS Syntax

Concept: A CSS rule consists of a selector and declaration block.

Example:

```
p { color: red; }
```

3. Selectors

Concept: Selectors target HTML elements. Types include element, class, ID, attribute, pseudo-class, and pseudo-element selectors.

Example:

```
.card { padding:20px; } #header { background:black; } a:hover { color:red; }
```

4. Colors and Backgrounds

Concept: CSS supports named colors, HEX, RGB, RGBA, HSL, gradients, and background images.

Example:

```
body { background: linear-gradient(to right, blue, purple); }
```

5. CSS Units

Concept: Common units include px, %, em, rem, vh, vw, fr.

Example:

```
.container { width:80%; font-size:1.2rem; }
```

6. Text Styling

Concept: Control alignment, decoration, spacing, shadows, and transformations.

Example:

```
h1 { text-transform: uppercase; text-shadow:2px 2px 4px gray; }
```

7. Fonts

Concept: Manage typography using font-family, size, weight, style, and web fonts.

Example:

```
body { font-family: Arial, sans-serif; }
```

8. Links and Lists

Concept: Style links using pseudo-classes and customize list markers.

Example:

```
a:hover { color:green; } ul { list-style-type:square; }
```

9. Tables

Concept: Tables can be styled using borders, spacing, zebra stripes, and hover effects.

Example:

```
table { border-collapse: collapse; }
```

10. Borders

Concept: Borders define the edge of an element.

Example:

```
.box { border:2px solid black; border-radius:10px; }
```

11. Margin and Padding

Concept: Margins create outer spacing; padding creates inner spacing.

Example:

```
.card { margin:20px; padding:15px; }
```

12. Box Model

Concept: Every element consists of content, padding, border, and margin.

Example:

```
* { box-sizing:border-box; }
```

13. Outline

Concept: Outlines are drawn outside borders and do not affect layout.

Example:

```
.input { outline:2px solid blue; }
```

14. Width and Height

Concept: Define dimensions and responsive constraints.

Example:

```
.box { width:300px; max-width:100%; }
```

15. Display Property

Concept: Controls how elements are rendered: block, inline, flex, grid, none.

Example:

```
.container { display:flex; }
```

16. Positioning

Concept: Use static, relative, absolute, fixed, and sticky positioning.

Example:

```
.menu { position:sticky; top:0; }
```

17. Overflow and z-index

Concept: Control overflowing content and stacking order.

Example:

```
.box { overflow:auto; z-index:10; }
```

18. Float and Clear

Concept: Older layout technique used before Flexbox/Grid.

Example:

```
.img { float:left; }
```

19. Flexbox

Concept: One-dimensional layout system for rows and columns.

Example:

```
.row { display:flex; justify-content:space-between; align-items:center; }
```

20. CSS Grid

Concept: Two-dimensional layout system for complex layouts.

Example:

```
.grid { display:grid; grid-template-columns:repeat(3,1fr); gap:20px; }
```

21. Transforms

Concept: Move, rotate, scale, and skew elements.

Example:

```
.box:hover { transform:scale(1.1); }
```

22. Transitions

Concept: Create smooth property changes.

Example:

```
.btn { transition:all .3s ease; }
```

23. Animations

Concept: Animate elements using keyframes.

Example:

```
@keyframes slide { from {left:0;} to {left:200px;} }
```

24. Responsive Design

Concept: Adapt layouts for different devices using media queries.

Example:

```
@media(max-width:768px){ .row{flex-direction:column;} }
```

25. CSS Variables

Concept: Reusable custom properties.

Example:

```
:root { --primary:#3498db; }
```

26. CSS Functions

Concept: Useful functions include calc(), min(), max(), clamp().

Example:

```
width: calc(100% - 50px);
```

27. Shadows and Filters

Concept: Enhance UI with shadows and visual effects.

Example:

```
.card { box-shadow:0 4px 10px rgba(0,0,0,.2); }
```

28. Best Practices

Concept: Use reusable classes, responsive units, Flexbox/Grid, and meaningful naming conventions.

Example:

```
Follow BEM and mobile-first design.
```

Common CSS Interview Questions

- What is the CSS Box Model?
- Difference between relative and absolute positioning?
- Difference between Flexbox and Grid?
- What is specificity?
- What is z-index?
- Difference between em and rem?
- What are CSS variables?
- How do media queries work?